User Creation Process Flow

How each role is created, which table the entry lands in, and what gets stamped on it (guard, role, tenant, creator).

Quick Reference: Who Lives Where

Role	Table	Guard	Created By	Tenant-scoped?
Super Admin	`admins`	`admin`	Nobody (first row, seeded manually)	No
Super Admin Staff	`admins`	`admin`	A Super Admin	No
Agency (owner)	`agency_users`	`agency`	A Super Admin / Super Admin Staff	Yes
Agency Staff	`agency_users`	`agency`	The Agency owner (or another Agency Staff, if permitted)	Yes
Agency Client	`clients`	`client`	Agency owner or Agency Staff	Yes

Process 1: The First Super Admin (bootstrap)

There's no one to "create" the very first Super Admin — this row is seeded directly during setup (e.g. a database seeder or one-time CLI command), not through the app's UI.

Table written: `admins`

sql

```sql
INSERT INTO admins (uuid, name, email, password, admin_role, created_by, status
VALUES (
  UUID(),
  'Platform Owner',
  'owner@yourapp.com',
  '<bcrypt_hash>',
  'super_admin',
  NULL,           -- no creator; this is the root account
  'active',
  NOW(), NOW()
);
```

Column	Value	Why
`admin_role`	`super_admin`	Marks this as the top-level role
`created_by`	`NULL`	Nothing created this account — it's the root

Process 2: Super Admin Creates a Super Admin Staff

Trigger: Super Admin, logged in via the `admin` guard, submits "Add Staff" form.

Table written: `admins` (same table as Super Admin — just a different `admin_role`)

```sql
INSERT INTO admins (uuid, name, email, password, admin_role, created_by, status
VALUES (
  UUID(),
  'Jane Support',
  'jane@yourapp.com',
  '<bcrypt_hash>',
  'super_admin_staff',    -- <-- the "type" is stored here
  1,                      -- id of the Super Admin who created this row
  'active',
  NOW(), NOW()
);
```

Then, assign the matching Spatie role (so permission checks work):

```sql
INSERT INTO model_has_roles (uuid, role_id, model_type, model_id)
VALUES (
  UUID(),
  (SELECT id FROM roles WHERE name = 'super_admin_staff' AND guard_name = 'admi
  'App\\Models\\Admin',
  <new_admin_id>
);
```

Column	Value	Why
`admin_role`	`super_admin_staff`	This is what marks the "type" — same table, different value
`created_by`	Super Admin's `id`	Traces who created this staff account
`model_has_roles.model_type`	`App\Models\Admin`	Tells Spatie this role applies to the `admins` table

Process 3: Super Admin (or Staff) Creates a New Agency

This is the biggest flow — it creates a **tenant** and its **owner login** together. Four tables get written, in this order:

```
1. tenants          — the tenant itself
2. agency_users     — the Agency's own login (agency_role = 'agency')
3. agencies         — business details, links tenant <-> owner account
4. domains          — the subdomain the tenant will be reached at
```

Step 1 — Create the tenant

sql

```sql
INSERT INTO tenants (uuid, name, slug, status, created_at, updated_at)
VALUES (UUID(), 'Acme Recruiting', 'acme', 'active', NOW(), NOW());
-- capture the new tenant's id, e.g. tenant_id = 5
```

Step 2 — Create the Agency's own login account

sql

```sql
INSERT INTO agency_users (
  uuid, tenant_id, name, email, password, agency_role,
  created_by_type, created_by_id, status, created_at, updated_at
) VALUES (
  UUID(),
  5,                    -- tenant_id from step 1
  'Acme Owner',
  'owner@acme.com',
  '<bcrypt_hash>',
  'agency',             -- <-- "type" stored here: this row IS the agency
  'admin',              -- polymorphic: creator is from the admins table
  1,                    -- id of the Super Admin (or Staff) who created it
  'active', NOW(), NOW()
);
-- capture the new agency_user's id, e.g. agency_user_id = 10
```

Step 3 — Create the business record linking tenant + owner

sql

```sql
INSERT INTO agencies (uuid, tenant_id, name, owner_agency_user_id, created_by_a
VALUES (UUID(), 5, 'Acme Recruiting', 10, 1, 'active', NOW(), NOW());
```

Step 4 — Create the tenant's subdomain

sql

```sql
INSERT INTO domains (uuid, tenant_id, domain, type, is_primary, ssl_status, cre
VALUES (UUID(), 5, 'acme.yourapp.com', 'subdomain', true, 'active', NOW(), NOW(
```

Step 5 — Assign the Spatie role

sql

```sql
INSERT INTO model_has_roles (uuid, role_id, model_type, model_id)
VALUES (
  UUID(),
  (SELECT id FROM roles WHERE name = 'agency' AND guard_name = 'agency'),
  'App\\Models\\AgencyUser',
  10
);
```

Resulting state after this process

Table	Row created	Key values
`tenants`	1	`id=5`, `slug=acme`
`agency_users`	1	`id=10`, `tenant_id=5`, `agency_role=agency`, `created_by_type=admin`, `created_by_id=1`
`agencies`	1	`tenant_id=5`, `owner_agency_user_id=10`, `created_by_admin_id=1`
`domains`	1	`tenant_id=5`, `domain=acme.yourapp.com`
`model_has_roles`	1	links `agency_users.id=10` to the `agency` role

Process 4: Agency Owner Creates an Agency Staff Member

Trigger: The Agency owner (or a Staff member with permission), logged in via the `agency` guard on their own subdomain, submits "Add Staff."

Table written: `agency_users` (same table as the Agency owner — different `agency_role`, different `tenant_id` scope already established by whoever is logged in)

sql

```sql
INSERT INTO agency_users (
  uuid, tenant_id, name, email, password, agency_role,
  created_by_type, created_by_id, status, created_at, updated_at
) VALUES (
  UUID(),
  5,                    -- SAME tenant_id as the Agency (inherited from logged
  'Staff Member',
  'staff@acme.com',
  '<bcrypt_hash>',
  'agency_staff',       -- <-- "type" stored here
  'agency_user',        -- polymorphic: creator is from agency_users table thi
  10,                   -- id of the Agency owner who created this row
  'active', NOW(), NOW()
);
-- capture new id, e.g. agency_user_id = 11
```

sql

```sql
INSERT INTO model_has_roles (uuid, role_id, model_type, model_id)
VALUES (
  UUID(),
  (SELECT id FROM roles WHERE name = 'agency_staff' AND guard_name = 'agency'),
  'App\\Models\\AgencyUser',
  11
);
```

Column	Value	Why
`agency_role`	`agency_staff`	Distinguishes from the owner row, same table
`tenant_id`	`5`	Always copied from whoever is logged in — never picked by the form
`created_by_type`	`agency_user`	This time the creator is another row in the SAME table, not an admin
`created_by_id`	`10`	The Agency owner's own `id`

> Optionally, add a `staff_profiles` row right after (degree, phone, designation, etc.) — see the earlier staff tables doc.

Process 5: Agency (or Staff) Creates a Client

Trigger: Agency owner or Agency Staff, logged in via the `agency` guard, submits "Add Client."

Table written: `clients` (separate table/guard — not `agency_users`, since a client never logs into the back office)

sql

```sql
INSERT INTO clients (uuid, tenant_id, name, email, password, created_by, status
VALUES (
  UUID(),
  5,                  -- SAME tenant_id, inherited from logged-in context
  'Client Contact',
  'contact@clientco.com',
  '<bcrypt_hash>',
  11,                 -- id of the Agency Staff (or owner) who created it
  'active', NOW(), NOW()
);
-- capture new id, e.g. client_id = 20
```

```sql
INSERT INTO model_has_roles (uuid, role_id, model_type, model_id)
VALUES (
  UUID(),
  (SELECT id FROM roles WHERE name = 'agency_client' AND guard_name = 'client'),
  'App\\Models\\Client',
  20
);
```

Optionally follow up with a `client_profiles` row (company name, industry, contact person) — see the client tables doc.

Full Flow at a Glance

```
Super Admin (seeded)
  |
  ├─ creates ──► Super Admin Staff        [admins, admin_role='super_admin_staff']
  |
  └─ creates ──► New Agency (tenant setup)
                   |
                   ├─ tenants             [1 row: the tenant]
                   ├─ agency_users        [1 row: agency_role='agency']
                   ├─ agencies            [1 row: business details]
                   └─ domains             [1 row: subdomain]
                        |
                        └─ Agency owner logs in, then:
                             |
                             ├─ creates ──► Agency Staff   [agency_users, agency_role='agency_staff']
                             |
                             └─ creates ──► Client         [clients, separate table/guard]
```

The pattern to remember:

- **Same table, different "type" column** → Super Admin / Super Admin Staff both in `admins` (`admin_role`); Agency / Agency Staff both in `agency_users` (`agency_role`).
- **Different table entirely** → Clients go in `clients`, not `agency_users`, because they need a completely separate login guard.
- `tenant_id` **is always inherited**, never chosen — it comes from whoever is logged in (or, for a brand-new Agency, from the tenant just created in the same transaction).
- `created_by` **traces the chain** — polymorphic where the creator could be from two different tables (`agency_users.created_by_type/id`), a simple FK where it's always

from one table (`clients.created_by` → always `agency_users.id`).